

A: RESEARCH QUESTION

Research question

Can we utilize the random forest regressor model to predict the factors influencing BMW Sales volume between 2010 and 2024 across various regions?

Justification of the research question

The primary objective of this data analysis is to determine whether the features of vehicles affect their sales volume. I will build a random forest regressor machine learning model and use it to predict if the vehicle features influence the sales volume of the vehicle. The benefit of this data analysis is that the motor vehicle sales industry can use it to determine whether incorporating specific vehicle features can increase vehicle sales.

Description of context

In this analysis, I will utilize the random forest regressor machine learning model to predict whether the sales volume of BMW vehicles from 2010 to 2024 is influenced by vehicle features.

The data will be obtained from the Kaggle website

(<https://www.kaggle.com/datasets/ahmadrazakashif/bmw-worldwide-sales-records-20102024>).

Hypothesis

For the purpose of this analysis, the following null and alternative hypotheses will be utilized.

Null hypothesis-

The sales volume for the various BMW vehicles is not significantly influenced by the various factors such as the model, color, fuel_type, transmission, Engine_Size_L, Mileage_KM, Price_USD in the various regions from the year 2010 to the year 2024.

Alternate Hypothesis-

The sales volume for the various BMW vehicles is significantly influenced by various factors, such as the model, color, fuel_type, transmission, Engine_Size_L, Mileage_KM, and Price_USD in the various regions from the year 2010 to the year 2024

B: DATA COLLECTION

The BMW worldwide sales records (2010-2024) data, which will be utilized for this data analysis, was obtained from the Kaggle.com website at this link:

<https://www.kaggle.com/datasets/ahmadrazakashif/bmw-worldwide-sales-records-20102024>.

The dataset comprises 50,000 sales records spanning the years 2010 to 2024. For the purpose of analysis, the features below will be utilized for analysis

1. Sales Volume

The sales volume represents the number of vehicles sold in each region annually from 2010 to 2024. The variable Sales Volume is a numeric variable and will be used as a response (dependent) variable. We shall be predicting the Sales Volume of the BMW vehicles

2. Model

The model represents the vehicle's product model line. There are 11 unique values. The models are 5 Series, i8, X3, 7 Series, M5, 3 Series, X1, M3, X5, i3, and X6. The variable "Model" is categorical and will be used as an independent variable.

3. Year

The year indicates the year the vehicles were sold, spanning from 2010 to 2024. The variable "Year" is numerical, and it will be used as a control variable.

4. Region

The Region shows the geographic market where the vehicle units were sold. There are 6 unique values consisting of Asia, North America, the Middle East, South America, Europe, and Africa.

The variable "Region" is categorical, and it will be used as a control variable.

5. Color

The color features show the BMW vehicle's exterior color and will be one of the factors analyzed to determine if it influences sales volume. There are 6 unique values consisting of Reds, Blue, Black, Silver, White, and Grey. The variable "Color" is categorical and will be used as an independent variable.

6. Fuel Type

The fuel type indicates the vehicle's fuel system and will be one of the factors analyzed to determine if it influences sales volume. There are 4 unique values: Petrol, Hybrid, Diesel, and Electric. The variable "fuel Type" is categorical and will be used as an independent variable.

7. Transmission

The transmission value specifies whether the vehicle has a manual or automatic transmission and will be one of the factors analyzed to determine if it influences sales volume. There are 2 unique values: Manual or transmission. The variable "Transmission" is categorical and will be used as an independent variable.

8. Engine_Size_L

The Engine_Size_L shows the Engine capacity in Liters and consists of 36 unique values. The variable "Engine_Size_L" is numerical and will be used as an independent variable.

9. Mileage_KM

Mileage_KM shows the distance driven in kilometers. The variable "Mileage_KM" is numerical and will be used as an independent variable.

10. Price_USD

Price_USD shows the price of the cars in US dollars. The variable "Price_USD" is numerical and will be used as an independent variable.

One advantage of this data is that it was readily available on a public website, and the fields were well described. The disadvantage of the data is that it did not contain customer demographic information, which would have enabled a more robust analysis to identify all the features that influence vehicle sales. I also lacked an understanding of how the field of sales classification was determined, and to address this issue, the feature was dropped as it did not describe a vehicle's characteristics.

C: DATA EXTRACTION AND PREPARATION

The data already exists on the Kaggle.com website and is available for download into a CSV file. I will download the dataset into a CSV file and then load it into a Python dataframe, from which I will explore the data and perform initial preprocessing steps to make the data usable for my model.

For the purpose of this data analysis, I will be utilizing the Python tool and the Random Forest regressor technique.

The random forest regressor technique is a machine learning algorithm that combines decision trees to improve predictive accuracy and make final predictions. The advantage of the random forest regressor method is that it can handle non-linear relationships and requires minimal tuning. The disadvantage of the random forest regressor model is that it can be computationally expensive

Python is an open-source general-purpose programming language with many library packages that can be used and shared with others. I utilized Python Programming Language because of its flexibility and extensive library collections, which are valuable for performing complex calculations and analysis. The disadvantage of the Python tool is that it can be inefficient with extremely large datasets.

The following libraries in Python will be utilized

- Pandas: Pandas will import the data from the CSV file, conduct exploratory data analysis, manipulate data, and convert categorical data to numerical data.
- Numpy: Numpy will be used for data manipulation.
- Matplotlib.pyplot: Matplotlib will be used for data visualization.
- Seaborn: Seaborn will be used for data visualization in the exploratory data analysis, such as creating heatmaps.
- `from sklearn.model_selection import train_test_split`: The function `train_test_split` from `sklearn.model_selection` will split the dataset into training and testing subsets.
- `from sklearn import preprocessing`: The library `preprocessing` from `sklearn` will be used for data preparation tasks, such as scaling, encoding categorical variables, and normalizing data.
- `from sklearn.feature_selection import SelectKBest`: `SelectKBest` from `sklearn` Feature selection will select the top k features from the dataset based on statistical tests, thereby reducing dimensionality and improving the model.
- `from sklearn.feature_selection import f_regression` : The function `f_regression` from `sklearn.feature_selection` will be used to perform the ANOVA F-test to select statistically significant features for regression tasks.

- from sklearn.ensemble import RandomForestRegressor: The library RandomForestRegressor from sklearn.ensemble will be used to implement the Random Forest algorithm for regression tasks
- from sklearn.model_selection import RandomizedSearchCV: The function RandomizedSearchCV from sklearn.model_selection will perform hyperparameter tuning by testing different parameter combinations and selecting the best ones for the Random Forest model.
- from sklearn.metrics import mean_squared_error : The function mean_squared_error from sklearn.metrics will be used to evaluate the performance of the regression model by measuring the average squared difference between predicted and actual values.
- from sklearn.metrics import r2_score : The function r2_score from sklearn.metrics will be used to evaluate the proportion of variance in the target variable explained by the Random Forest Regressor.
- from sklearn.metrics import mean_absolute_error: The mean_absolute_error function from sklearn.Metrics will be used to calculate the Mean Absolute Error (MAE) between predicted and actual values in a regression model.
- Warnings : Additional packages will be used to suppress future warnings and package changes expected from the library, ensuring a clean output during the analysis.

Data extraction

The following steps were taken for data extraction, exploration and cleaning.

- Importing the python libraries

```
# 1. Import libraries.
```

```
!pip install pandas
!pip install numpy
!pip install seaborn
!pip install matplotlib
!pip install --upgrade seaborn

#Data Manipulation / Handling data
import pandas as pd #For loading data, data exploration
import numpy as np # for data manipulation

#Data visualization
import matplotlib.pyplot as plt #For plotting feature importance
import seaborn as sns #For Exploratory data visualization to create correlation heatmaps

#Machine Learning
#Data Preprocessing
from sklearn.model_selection import train_test_split # for splitting data
from sklearn import preprocessing #forscaling and encoding

#Feature Selection
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_regression

#Modeling
from sklearn.ensemble import RandomForestRegressor

#Hyperparameter tuning
from sklearn.model_selection import RandomizedSearchCV

#Model Evaluation
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error

#Others
import warnings
with warnings.catch_warnings():
    warnings.simplefilter("ignore", FutureWarning)
```

- Loading the data into a dataframe

```
#Can we utilize the random forest regressor model to predict the factors influencing BMW Sales volume between  
#2010 and 2024 across various regions?
```

```
#File path variables
```

```
Local_path = 'C:/Users/beatr/OneDrive/Desktop/WGU/D214/Task1/Data/'  
InputFilePath = Local_path + 'BMW sales data (2010-2024).csv'  
OutputfilePath = Local_path + 'BMW_cleaned_df.csv'  
XtrainOutputfilePath = Local_path + 'Xtrain.csv'  
XtestOutputfilePath = Local_path + 'Xtest.csv'  
ytrainOutputfilePath = Local_path + 'ytrain.csv'  
ytestOutputfilePath = Local_path + 'ytest.csv'  
TargetVariable = "Sales_Volume"
```

```
# 2. Load the data.
```

```
#Loading the data  
#Churn_df = pd.read_csv(InputFilePath, index_col=0)  
BMW_df = pd.read_csv(InputFilePath)
```

- Initial data exploration

```
#Initial data exploration (Understanding the data)
```

```
pd.set_option('display.max_columns', None)  
BMW_df.head()
```

```
]:
```

	Model	Year	Region	Color	Fuel_Type	Transmission	Engine_Size_L	Mileage_KM	Price_USD	Sales_Volume	Sales_Classification
0	5 Series	2016	Asia	Red	Petrol	Manual	3.5	151748	98740	8300	High
1	i8	2013	North America	Red	Hybrid	Automatic	1.6	121671	79219	3428	Low
2	5 Series	2022	North America	Blue	Petrol	Automatic	4.5	10991	113265	6994	Low
3	X3	2024	Middle East	Blue	Petrol	Automatic	1.7	27255	60971	4047	Low
4	7 Series	2020	South America	Black	Diesel	Manual	2.1	122131	49898	3080	Low

```
BMW_df.shape
```

```
]:(50000, 11)
```

```
BMW_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 50000 entries, 0 to 49999  
Data columns (total 11 columns):  
#   Column                Non-Null Count  Dtype  
---  ---  
0   Model                  50000 non-null  object  
1   Year                   50000 non-null  int64  
2   Region                 50000 non-null  object  
3   Color                  50000 non-null  object  
4   Fuel_Type              50000 non-null  object  
5   Transmission            50000 non-null  object  
6   Engine_Size_L          50000 non-null  float64  
7   Mileage_KM             50000 non-null  int64  
8   Price_USD              50000 non-null  int64  
9   Sales_Volume           50000 non-null  int64  
10  Sales_Classification    50000 non-null  object  
dtypes: float64(1), int64(4), object(6)  
memory usage: 4.2+ MB
```


- Pre-processing the data. This includes checking for duplicates and removing them, checking for unique values, trimming whitespace, checking for correlation, removing features that will not be used in the analysis, one-hot encoding, scaling numeric features, and feature selection.

```
| # 3. Preprocessing (Cleaning) - includes removing outliers, white spaces, NULL/ NAN  
#values, imputing missing data, binding, one-hot encoding, etc.
```

```
| #Checking for Nulls and missing data
```

```
print(BMW_df[BMW_df[BMW_df.columns[BMW_df.isna().any()]].columns].info())
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 50000 entries, 0 to 49999  
Empty DataFrame  
None
```

```
| #Trim and clean white spaces
```

```
BMW_df=BMW_df.map(lambda x: x.strip() if isinstance(x,str)else x)
```

```
| #Checking for duplicated data
```

```
print("Duplicated rows")  
print(BMW_df.duplicated().any())  
print(" ")  
print("Duplicated columns")  
print(BMW_df.columns.duplicated().any())
```

```
Duplicated rows  
False
```

```
Duplicated columns  
False
```

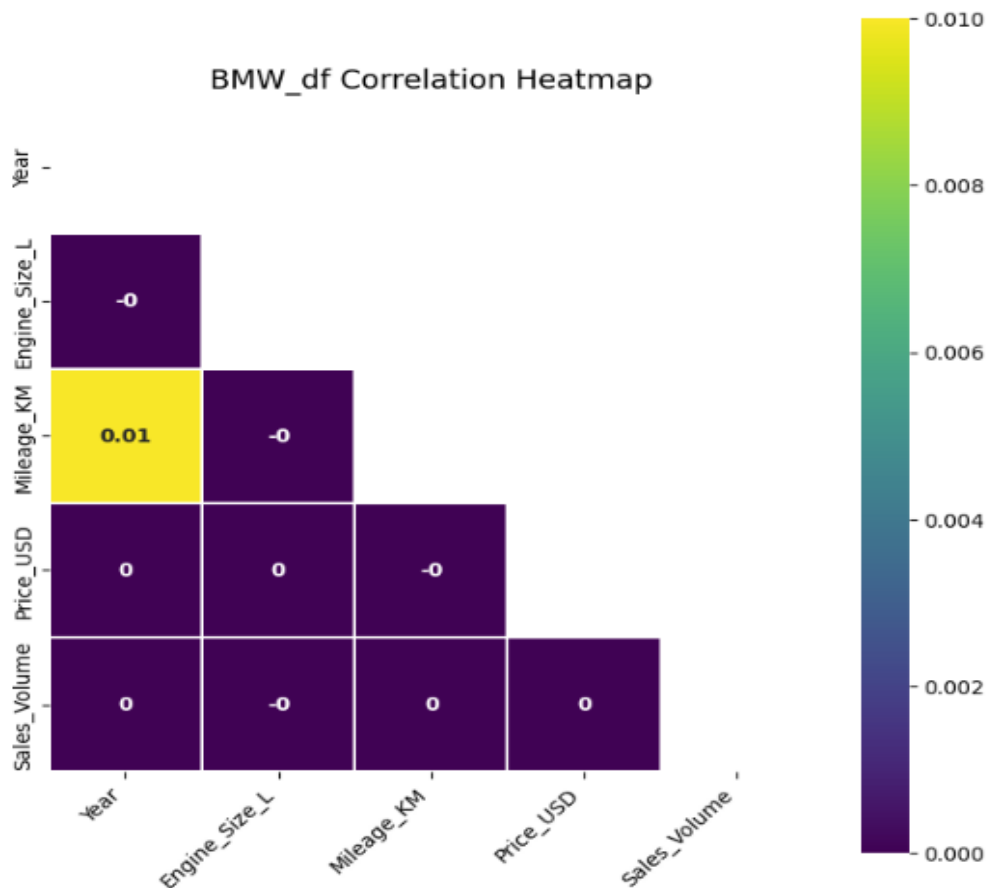
```
| #Checking for unique values
```

```
BMW_df.nunique()
```

```
: Model          11  
Year            15  
Region          6  
Color           6  
Fuel_Type       4  
Transmission    2  
Engine_Size_L   36  
Mileage_KM      44347  
Price_USD       38246  
Sales_Volume    9845  
Sales_Classification  2  
dtype: int64
```

```
#Analysing the Correlation /Relationship analysis
Continuous_columns = BMW_df.select_dtypes(include=['float64', 'int64']).columns.tolist()
Cor_data = BMW_df[Continuous_columns]

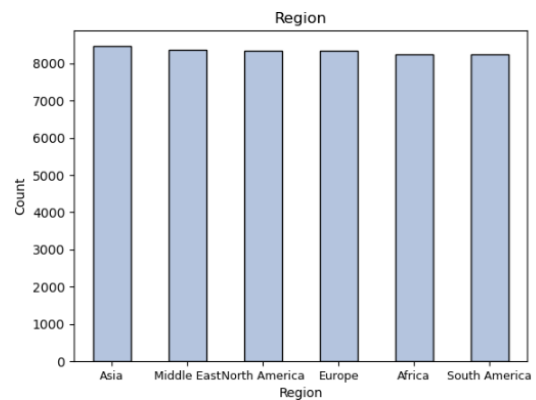
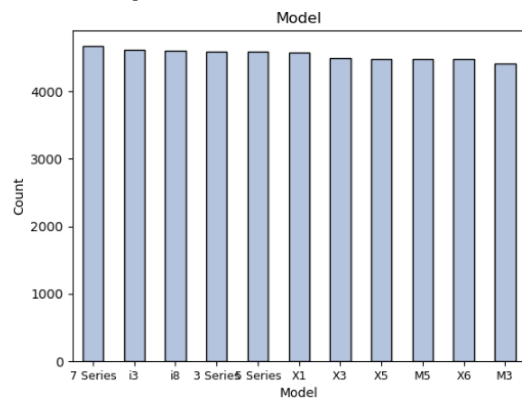
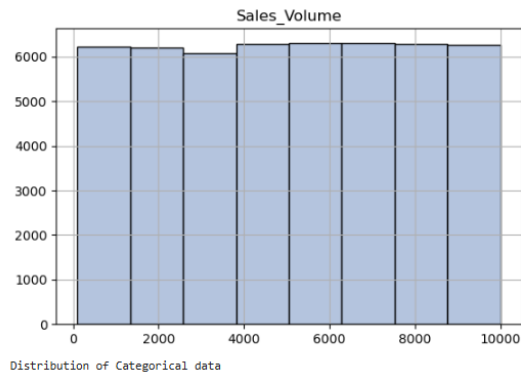
def Generate_heatmap_graph(corr, chart_title, mask_uppertri=False):
    """ Based on features, generate a correlation matrix """
    mask = np.zeros_like(corr, dtype=bool)
    mask[np.triu_indices_from(mask)] = mask_uppertri
    fig, ax = plt.subplots(figsize=(8, 8))
    sns.heatmap(corr,
                mask=mask,
                square=True,
                annot=True,
                annot_kws={'size': 10, 'rotation': 0, 'weight': 'bold'},
                cmap=plt.get_cmap('viridis'),
                linewidths=.1,
                ax=ax)
    ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right') # Rotate x-axis labels
    plt.title(chart_title, fontsize=14)
    plt.show()
var_corr = round(Cor_data.corr(), 2)
Generate_heatmap_graph(var_corr, chart_title='BMW_df Correlation Heatmap', mask_uppertri=True)
```



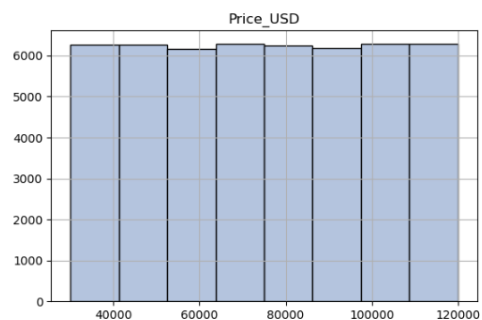
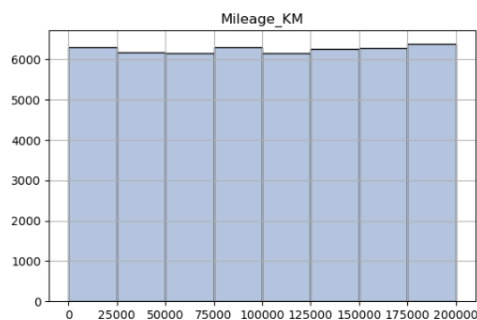
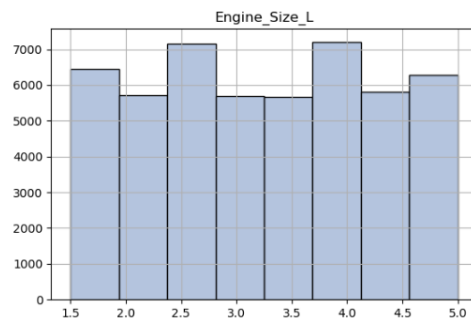
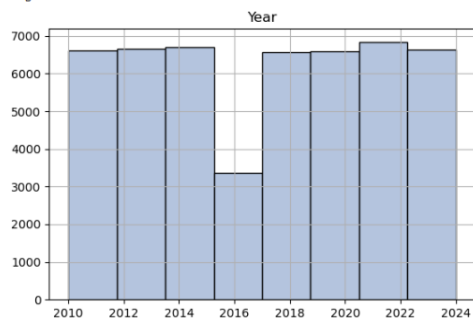
```
#Dropping Columns that will not be used in the analysis
BMW_df = BMW_df.drop (columns = ['Sales_Classification'])
```

Data visualization

D214: DataAnalyticsGraduateCapstoneProject_Task2_DataAnalytics_RandomForestRegressor
BeatriceM_WGU_MSDA11

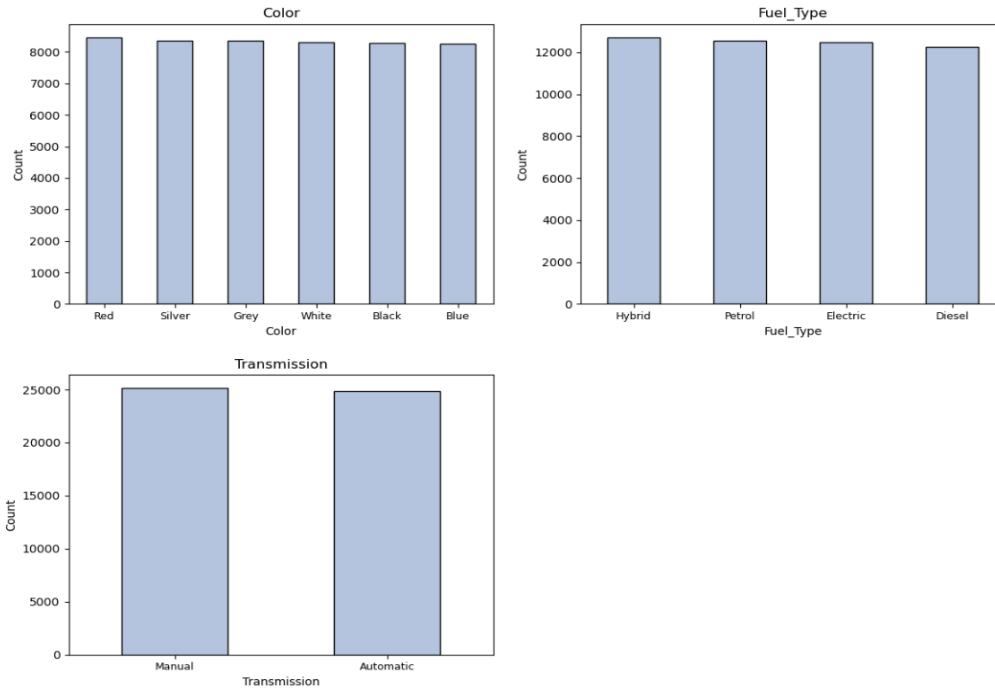


(Figure size 1000x2000 with 0 Axes)



D214: DataAnalyticsGraduateCapstoneProject_Task2_DataAnalytics_RandomForestRegressor

BeatriceM_WGU_MSDA12



- Converting the categorical data into numeric data

Converting Categorical data to numerical data through dummy encoding. Include all dummies for non-binary variables and include only one dummy variable for binary columns.

```
#Converting Categorical data to numerical data through dummy encoding (One Hot encoding)

#Dummy encoding is often used as an alternative term.
#However, dummy encoding typically drops one of the binary columns to prevent multicollinearity (unlike standard one-hot encoding).
#The method uses the option 'drop_first=True'.

categorical_columns = BMW_df.select_dtypes(include=['object']).columns.tolist()

# Separate binary and non-binary categorical columns
binary_columns = [col for col in categorical_columns if BMW_df[col].nunique() == 2]
non_binary_columns = [col for col in categorical_columns if BMW_df[col].nunique() > 2]

# Create dummy variables
BMW_df_encoded = pd.get_dummies(
    BMW_df,
    columns=non_binary_columns, # Include all dummies for non-binary variables
    dummy_na=False,
    drop_first=False, # Do not drop any dummy for non-binary
    dtype='int64'
)

# For binary columns, include only one dummy variable
BMW_df_encoded = pd.get_dummies(
    BMW_df_encoded,
    columns=binary_columns,
    dummy_na=False,
    drop_first=True, # Drop the first dummy for binary
    dtype='int64'
)

# Display the resulting dataframe
BMW_df_encoded.head()
```

	Year	Engine_Size_L	Mileage_KM	Price_USD	Sales_Volume	Model_3 Series	Model_5 Series	Model_7 Series	Model_M3	Model_M5	Model_X1	Model_X3	Model_X5	Model_X6	Model_X8	Region_Africa	Region_Asia	Region_Europe	Region_Middle East	Region_North America	Region_South America	Color_Black	Color_Blue	Color_Grey	Color_Red	Color
0	2016	3.5	151748	98740	8300	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	
1	2013	1.6	121671	79219	3428	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	
2	2022	4.5	10991	113265	6994	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	
3	2024	1.7	27255	60971	4047	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	1	0	0	
4	2020	2.1	122131	49898	3080	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	

D214: DataAnalyticsGraduateCapstoneProject_Task2_DataAnalytics_RandomForestRegressor

BeatriceM_WGU_MSDA13

```
: # variable for numeric data
numerical_features = BMW_df_encoded.select_dtypes(include="number").columns
print("numerical_features")
for feature in numerical_features:
    print(feature)

numerical_features
Year
Engine_Size_L
Mileage_KM
Price_USD
Sales_Volume
Model_3 Series
Model_5 Series
Model_7 Series
Model_M3
Model_M5
Model_X1
Model_X3
Model_X5
Model_X6
Model_i3
Model_i8
Region_Africa
Region_Asia
Region_Europe
Region_Middle East
Region_North America
Region_South America
Color_Black
Color_Blue
Color_Grey
Color_Red
Color_Silver
Color_White
Fuel_Type_Diesel
Fuel_Type_Electric
Fuel_Type_Hybrid
Fuel_Type_Petrol
Transmission_Manual
```

- Standardization of the numerical columns

Scaling the numerical features to allow for analysis

```
: # 5. Standardization (includes scaling, normalization, data types, etc., as applicable) - Not Applicable
```

```
: # Separate the X variables (explanatory) from the Y variable (response) into different dataframes
df_X = BMW_df_encoded.drop([TargetVariable], axis=1).copy()
df_Y = BMW_df_encoded[TargetVariable].copy()
```

```
: # Standardize the X variables (explanatory) to put all on the same scale for Random forest regressor
df_X = pd.DataFrame(preprocessing.MinMaxScaler().fit_transform(df_X), columns=df_X.columns)
df_X.head()
```

	Year	Engine_Size_L	Mileage_KM	Price_USD	Model_3 Series	Model_5 Series	Model_7 Series	Model_M3	Model_M5	Model_X1	Model_X3	Model_X5	Model_X6	Model_i3	Model_i8	Region_Africa	Region_Asia	Region_Europe	Region_Middle East	Region_North America	Region_South America	Color_Black	Color_Blue	Color_Grey	Color_Red	Color_Silver
0	0.428571	0.571429	0.758752	0.763795	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
1	0.214286	0.028571	0.608361	0.546890	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0
2	0.857143	0.857143	0.054942	0.925187	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0
3	1.000000	0.057143	0.136265	0.344130	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0
4	0.714286	0.171429	0.610661	0.221094	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	0.0

- Feature selection

Feature selection using SelectKBest involves selecting the top 15 features based on a statistical test to improve model performance and reduce overfitting.

```
# 6. Feature Selection

#Identify the optimal number of features to select
k =min(15, df_X.shape[1])
selector = SelectKBest(score_func=f_regression, k=k)
X_selected = selector.fit_transform(df_X, df_y)

# Get the selected feature names
Selected_Features = df_X.columns[selector.get_support()]
print("Selected features:")
for feature in Selected_Features:
    print(feature)

Selected features:
Engine_Size_L
Model_5 Series
Model_7 Series
Model_X1
Model_i3
Region_Africa
Region_Europe
Region_North America
Region_South America
Color_Black
Color_Grey
Color_Silver
Color_White
Fuel_Type_Diesel
Fuel_Type_Petrol
```

- Reviewing the cleaned data

```
# 7. Export the Cleaned Data

# Recombine df_X (feature set) and df_y (response variable) to save the whole thing in one dataframe
Churn_Cleaned_df = df_X[Selected_Features].copy() # Only keep the selected features
Churn_Cleaned_df[TargetVariable] = df_y.reset_index(drop=True)

# Save dataframe to CSV, ignore ind

print('The Cleaned Data')
print(f'Churn_Cleaned_df.shape: {Churn_Cleaned_df.shape}')
Churn_Cleaned_df.info()

The Cleaned Data
Churn_Cleaned_df.shape: (50000, 16)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Engine_Size_L          50000 non-null   float64
1   Model_5 Series         50000 non-null   float64
2   Model_7 Series         50000 non-null   float64
3   Model_X1               50000 non-null   float64
4   Model_i3               50000 non-null   float64
5   Region_Africa          50000 non-null   float64
6   Region_Europe          50000 non-null   float64
7   Region_North America   50000 non-null   float64
8   Region_South America   50000 non-null   float64
9   Color_Black            50000 non-null   float64
10  Color_Grey             50000 non-null   float64
11  Color_Silver           50000 non-null   float64
12  Color_White            50000 non-null   float64
13  Fuel_Type_Diesel       50000 non-null   float64
14  Fuel_Type_Petrol       50000 non-null   float64
15  Sales_Volume           50000 non-null   int64
dtypes: float64(15), int64(1)
memory usage: 6.1 MB
```

D: ANALYSIS

In doing my analysis, the following steps and processes were carried out

Splitting the data into training and testing sets with a test size of 20% and defining the random seed. The train_test_split function from pandas was used to split the data. Split the data into training and test data sets using train_test_split.

The data will be split into X_train, X_test, y_train, and y_test to separate the training data used to build the model from the testing data used to evaluate the model's performance. This will ensure the model is trained and validated on distinct datasets, preventing overfitting and generalizing well to new data.

```
# 8. Split the Data - split the cleaned data.
```

```
# Split the data into training and test sets (80% training, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X_selected, df_y, test_size=0.2, random_state=42)
```

```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(40000, 15)
(10000, 15)
(40000,)
(10000,)
```

```
# Convert X_train, X_test, y_train, and y_test into DataFrames or Series
X_train_df = pd.DataFrame(X_train, columns=Selected_Features)
X_test_df = pd.DataFrame(X_test, columns=Selected_Features)
y_train_df = pd.Series(y_train)
y_test_df = pd.Series(y_test)
```

```
# Save each of the split data for Xtrain,Xtest,ytrain,y,test
X_train_df.to_csv(XtrainOutputfilePath, index=False)
X_test_df.to_csv(XtestOutputfilePath, index=False)
y_train_df.to_csv(ytrainOutputfilePath, index=False)
y_test_df.to_csv(ytestOutputfilePath, index=False)
```

- Initializing the model

The RandomForestRegressor model was initialized. The random forest regressor technique is a machine learning algorithm that combines decision trees to improve predictive accuracy and make final predictions. The advantage of the random forest regressor method is that it can handle non-linear relationships and requires minimal tuning. The disadvantage of the random forest regressor model is that it can be computationally expensive. The initial model, Random Forest regressor model, was created with default hyperparameters, and the random state was set to 42 to ensure reproducibility. The model was then fit into the training datasets.


```
# 9. Create or initialize the model - call the RandomForestRegressor
```

```
initial_model = RandomForestRegressor(random_state=42)
```

```
# 10. Train and fit the model.
```

```
initial_model.fit(X_train, y_train)
```

```
: RandomForestRegressor  
RandomForestRegressor(random_state=42)
```

- Evaluating the initial model

The model was evaluated using the metrics R^2 score, MSE, RMSE, and MAE

R^2 score measures how well a regression model explains the variation in the target variable.

With 1 meaning perfect prediction, 0 meaning that the model doesn't predict better than the mean, and negative values show that the model's performance is worse than predicting with the mean.

Mean Squared Error (MSE) is a metric that evaluates the performance of a regression model by measuring the average squared difference between predicted and actual values. MSE quantifies the overall error of the model in predicting the target variable.

Root Mean Squared Error (RMSE) is a metric that evaluates the accuracy of a predictive model by measuring the average magnitude of the prediction errors.

MAE is a regression metric that measures the average magnitude of errors without squaring them, making it less sensitive to outliers.

Based on the initial R^2 score of -0.1797 , the high MSE of 9639368, RMSE of 3104, and MAE of 2624, the results indicate that there is no correlation between the vehicle features and the sales volume

```
11]: # 11. Evaluate the model for Accuracy and check the error metrics

12]: y_pred_initial = initial_model.predict(X_test)
mse_initial = mean_squared_error(y_test, y_pred_initial)
r2_initial = r2_score(y_test, y_pred_initial)
print(f"Initial Model - Mean Squared Error: {mse_initial}")
print(f"Initial Model - R^2 Score: {r2_initial}")
rmse_initial = mse_initial ** 0.5
mae_initial = mean_absolute_error(y_test, y_pred_initial)

print(f"Initial Model - Root Mean Squared Error (RMSE): {rmse_initial}")
print(f"Initial Model - Mean Absolute Error (MAE): {mae_initial}")

Initial Model - Mean Squared Error: 9639368.307659732
Initial Model - R^2 Score: -0.17965989962469298
Initial Model - Root Mean Squared Error (RMSE): 3104.733210383741
Initial Model - Mean Absolute Error (MAE): 2624.0512663062073
```

- Optimize the Model

RandomizedSearchCV was used to optimize the model. RandomizedSearchCV from the library scikit-learn is a hyperparameter tuning technique in machine learning that searches for the best combination of hyperparameters for a model by randomly sampling from a predefined set of possible values, where each sampled combination is evaluated using a cross-validation, which allows a more reliable performance estimate when one does not know which hyperparameters to use. The hyperparameters below were used.

- `n_estimators` : The `n_estimators` define the number of decision trees in the forest. The maximum number of estimators in this analysis is a range that consists of 100, 200 and 400.
- `Max_depth`: `Max_depth` controls the number of trees. The maximum depth in this analysis ranges from 0 to 10 and 20.
- `min_samples_split`: The `min_samples_split` specifies the minimum number of samples required to split an internal node. In this analysis, it ranges from 2, 5, and 10
- `min_samples_leaf`: The `min_samples_leaf` determines the minimum number of samples that must be present in a leaf node. The values range from 1, 2, 4

- `max_features`: The `max_features` controls the number of features considered when splitting a node. The values are `sqrt` and `log2`, which help introduce randomness and reduce correlation between trees.
- `Bootstrap`: Bootstrapping introduces randomness, helping reduce overfitting, and the value was set to `true`.

The final model used the values below for the hyperparameters

- `max_depth=10`,
- `max_features = 'log2'`,
- `min_samples_leaf=4`,
- `n_estimators=400`,
- `random_state=42`
- `bootstrap = True`

```
: # 12. Optimize the model (includes hyperparameter tuning ).

: param_dist = {
:     'n_estimators': [100, 200, 400],
:     'max_depth': [None, 10, 20],
:     'min_samples_split': [2, 5, 10],
:     'min_samples_leaf': [1, 2, 4],
:     'max_features': [ 'sqrt', 'log2'],
:     'bootstrap': [True]
: }

: random_search = RandomizedSearchCV(
:     estimator=RandomForestRegressor(random_state=42),
:     param_distributions=param_dist,
:     n_iter=20,
:     cv=3,
:     scoring='r2',
:     verbose=1,
:     random_state=42,
:     n_jobs=-1
: )
: random_search.fit(X_train, y_train)

: best_params = random_search.best_params_
: final_model = random_search.best_estimator_
: print("Best_Params")
: print(best_params)
: print("final_model")
: print(final_model)

Fitting 3 folds for each of 20 candidates, totalling 60 fits
Best_Params
{'n_estimators': 400, 'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2', 'max_depth': 10, 'bootstrap': True}
final_model
RandomForestRegressor(max_depth=10, max_features='log2', min_samples_leaf=4,
                       n_estimators=400, random_state=42)
```

- Evaluating the final model

The final model was evaluated using the metrics R^2 score, MSE, RMSE, and MAE.

The final model improved slightly after optimization, with an R^2 Score of -0.0036 , a high MSE of 8200764.15, an RMSE of 2863.70, and an MAE of 2479.33. The results indicate that there is no correlation between the vehicle features and the sales volume

```
➤ y_pred_final = final_model.predict(X_test)

mse_final = mean_squared_error(y_test, y_pred_final)
r2_final = r2_score(y_test, y_pred_final)
rmse_final = np.sqrt(mse_final)
mae_final = mean_absolute_error(y_test, y_pred_final)

print("Final (Optimized) Model Performance:")
print(f"Mean Squared Error: {mse_final:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_final:.2f}")
print(f"Mean Absolute Error (MAE): {mae_final:.2f}")
print(f" $R^2$  Score: {r2_final:.4f}")

Final (Optimized) Model Performance:
Mean Squared Error: 8200764.15
Root Mean Squared Error (RMSE): 2863.70
Mean Absolute Error (MAE): 2479.33
 $R^2$  Score: -0.0036
```

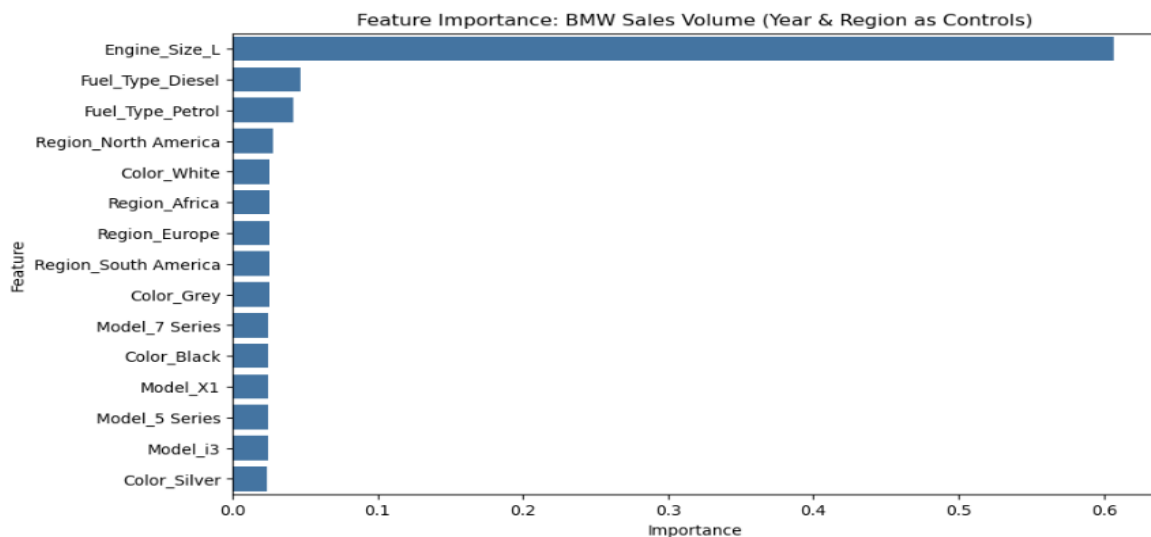
- Feature importance visualization

When creating a feature importance visualization, the visual indicates that

Engine_Size_L has a minimal influence on the sales volume. All other features do not appear to have any influence on sales volume.

```
In [ ]: feature_importance = pd.DataFrame({
        'Feature': Selected_Features,
        'Importance': final_model.feature_importances_
    }).sort_values(by='Importance', ascending=False)

plt.figure(figsize=(10,6))
sns.barplot(x='Importance', y='Feature', data=feature_importance)
plt.title('Feature Importance: BMW Sales Volume (Year & Region as Controls)')
plt.show()
```



The code



BeatriceMwangi_D2BeatriceMwangi_D2
14_CAPSTONE_BMW14_CAPSTONE_BMW

E: DATA SUMMARY AND IMPLICATIONS

The primary objective of this data analysis is to determine whether the features of vehicles affect the sales volume using the random forest regressor model, based on the BMW dataset from 2010 to 2024. The features model, color, fuel_type, transmission, Engine_Size_L, Mileage_KM, Price_USD, in the various regions with the Year as a control variable, were evaluated to determine if they had a significant influence on the sales volume. The results of the initial model, which includes an R^2 Score of -0.1797 , a high MSE of 9639368, an RMSE of 3104, and an MAE of 2624, and the final model had an R^2 Score of -0.0036 , a high MSE of 8200764.15, an RMSE of 2863.70, and an MAE of 2479.33. The results indicate poor model performance, characterized by a negative R^2 value, a large RMSE, and a high MSE score. The model suggests that vehicle features have little to no impact on sales volume. The model hence supports the null hypothesis, which is as below

The sales volume for the various BMW vehicles is not significantly influenced by the various factors such as the model, color, fuel_type, transmission, Engine_Size_L, Mileage_KM, Price_USD in the various regions from the year 2010 to the year 2024.

One limitation of the data analysis is that the dataset only contains the features of a vehicle, and no other variables, such as customer demographics and economic conditions, that may impact sales volume. Additionally, the vehicle's features have little or no impact on influencing buying decisions on their own.

The recommendation to the executive of the BMW company is that the vehicle features alone have no impact on determining the sales volume of various vehicles from 2010 to 2024 in

different regions, and they may need to consider the economic conditions and customer demographics.

Based on the results, which uphold the null hypothesis, it is shown that the BMW sales volume is not influenced by the vehicle's features, for future analysis, I recommend that the dataset also include customers' demographics, competition, and market conditions to determine which features have the most significant impact on sales and explore other machine learning algorithms that may capture nonlinear patterns.

F: SOURCES

MSDA Capstone Topic Approval Form_Beatrice Mwangi

Beatrice Mwangi_Task2_D209_DataMining1_PerformanceAssessment

The data was obtained from the Kaggle.com website on the link below

<https://www.kaggle.com/datasets/ahmadrazakashif/bmw-worldwide-sales-records-20102024>

G: PROFESSIONAL COMMUNICATION